

PART V — TOKENS, CONTEXT, AND MODEL CHOICE

Chapter 24

Long-Context Performance and the Failure Cliff

“The context window does not fail by refusing a prompt that is too long. It fails quietly, by answering a shorter question than the one that was actually asked.”

Chapter 24 — Long-Context Performance and the Failure Cliff

"Lost in the middle" is the name commonly given to the empirical finding that transformer-based LLMs attend more reliably to information near the start and end of a long prompt than to information buried in the middle. This chapter grounds that general finding in three specific, project-real events: the '~16K prompt token' degradation onset ADR-005 identified for 'llama-3.1-8b-instant', the 11,133-character citation-repetition failure that motivated ADR-012's prompt restructuring, and BUG-F013's 40-iteration runaway loop that hit Groq's rate limit before the model itself ever "failed" in the conventional sense.

24.1 The lost-in-the-middle problem

Definition — Lost in the middle

The empirically observed tendency of transformer-based LLMs to attend more reliably to tokens near the beginning and end of a prompt than to tokens in the middle, independent of how relevant the middle content actually is to the task.

This is precisely the mechanism ADR-012 names as its rationale: "transformer attention shows recency bias — the last tokens before the user message receive the highest weight on the next-token decision." The fix this project actually shipped — moving `\_PROCESS\_INSTRUCTIONS` to the very end of the system prompt, after the thumbsdown history and blocked-variant injections that used to sit last — is a direct, working exploitation of this bias rather than an attempt to defeat it. Section 24.4 traces the specific failure this fix was built to stop.

The practical consequence for a RAG pipeline specifically: content placed in the geometric middle of a long context — often exactly where accumulated tool results and retrieved chunks sit, sandwiched between the system prompt and the current turn — is the content most at risk of being under-weighted by the model relative to its actual relevance. This is a second, independent argument for compression (Chapter 22) beyond the token-budget argument Chapter 23 made: fewer tokens in the middle is not just cheaper, it is also more reliably attended to.

24.2 Empirical zones for an 8B model

ADR-005 records a specific, measured onset point for 'llama-3.1-8b-instant': "its instruction-following degrades at ~16K prompt tokens." This is not a vendor specification — it is this project's own observation, made during Batch testing (Batches 6, 8, and 10) where "poor analytical reasoning, acronym confusion, weak synthesis" were traced back to 8B model limitations at scale. Below that threshold, the model's behavior in this project's own dry runs is reliable; approaching and past it, quality measurably declines.

A three-zone reliability model follows directly from that single data point plus the model's own architectural ceiling. Below ~16,000 prompt tokens, treat the model as reliable — the green zone this project's own testing supports directly. Between ~16,000 and the 70B model's 32,768-token output ceiling, treat the model as degrading — instruction-following softens, but the model has not yet run out of window. Above that, deep into territory only the total 128,000-token window technically permits, treat any single call as high-risk — a region this project's own testing never validated as safe to rely on for the 8B model specifically.

Attention is not flat across a long prompt

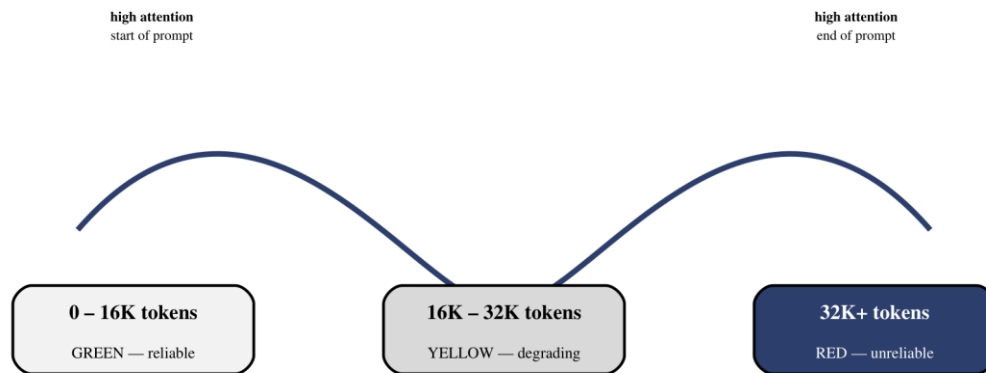


Figure 24.1 — Attention is highest at the edges of a prompt; the three reliability zones for the 8B model are grounded in ADR-005's ~16K degradation onset.

Figure 24.1's zone boundaries are a project-specific empirical finding, not a universal constant — a different model, a different prompt structure, or a different task would shift where green ends and red begins. What generalizes is the method: measure the actual degradation onset for the specific model in production, rather than assuming the vendor's advertised total window size is a reliability guarantee.

24.3 Where the rules start being ignored

ADR-013 names a second, more specific ceiling: "the 8B model's instruction-following ceiling (~1,800 tokens)" — the point past which thumbdown-history injection into the system prompt risks burying ``_PROCESS_INSTRUCTIONS`` far enough from the generation point that the recency-bias exploitation from Section 24.1 stops working. This is why ADR-013 recommends capping thumbdown injection to the most recent two records: not an arbitrary conservatism, but a number chosen against a measured instruction-following budget.

This 1,800-token figure is smaller than the ~16,000-token general degradation onset from Section 24.2 for a specific reason: instruction-following (correctly obeying "do NOT batch tool calls," for instance) is a narrower, more fragile capability than general reasoning or retrieval quality, and it degrades earlier under context growth than broader task competence does. A model can still retrieve reasonably well at 10,000 tokens of context while simultaneously beginning to ignore a specific procedural constraint stated 1,800 tokens back from the generation point.

24.4 Hallucination vs. instruction drift vs. runaway loops

Three distinct failure modes are easy to conflate under the single label "the model got worse," but this project's own bug ledger shows they have different triggers and different fixes, and treating them as one problem risks applying the wrong remedy.

Hallucination under context pressure looks like the pre-ADR-020 behavior: the 8B model calling ``check_answer_quality`` in Iteration 1, before any retrieval had occurred, with an answer synthesized from training knowledge rather than retrieved chunks. This is not a context-length problem at all — it happened at minimal context size — but a tool-availability problem, fixed by removing the option entirely rather than by trying to word a constraint the model would reliably obey.

Instruction drift is a genuine context-pressure symptom: the 11,133-character citation-repetition degeneration Status.md records, where context dilution caused the model to "vomit citations in a loop" despite output-format rules stated in the system prompt. ADR-012's prompt restructuring — moving ``_PROCESS_INSTRUCTIONS`` last — targets exactly this failure mode.

Runaway loops are a third, structurally different failure: BUG-F013's 40-plus-iteration session that exhausted Groq's 6,000-token-per-minute rate limit. This is not the model disobeying an instruction — the model was doing exactly what an unconstrained loop invited it to do, generate another query and try again. The fix was architectural ('MAX_ITERATIONS', 'MAX_TOTAL_RETRIEVALS', in-batch dedup), not a prompt change, because no wording in a system prompt reliably stops a small model from continuing a pattern that the system itself never told it to stop.

Figure 24.2 places all three failure modes side by side specifically so the differing fix column is visible at a glance: a schema change, a prompt restructure, and a hard iteration cap are not interchangeable remedies, and applying the wrong one to a given symptom wastes an engineering cycle without addressing the actual root cause.

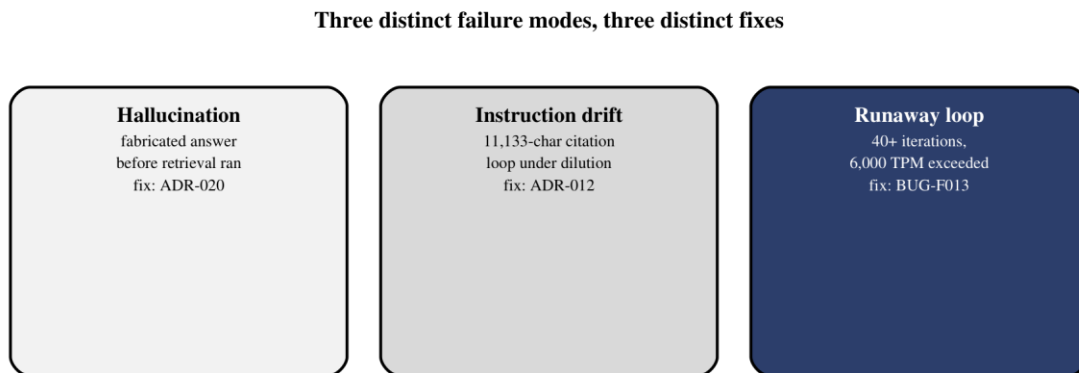


Figure 24.2 — Hallucination, instruction drift, and runaway loops are three distinct failures with three distinct, independently-verified fixes.

The practical diagnostic question when a run produces a bad answer is which of these three columns the symptom belongs to. An answer that cites facts absent from any retrieved chunk, appearing early in a session before meaningful retrieval occurred, points to Figure 24.2's leftmost column — a tool-schema problem. An answer that violates a stated output-format rule despite the rule being present somewhere in the prompt points to the middle column — a prompt-ordering problem, addressed in Chapter 26. A session that simply never terminates, or terminates only after burning far more iterations than the query complexity warrants, points to the rightmost column — a missing or miscalibrated guardrail, addressed further in Chapter 27.

Common pitfall — Treating every quality regression as a prompt problem

Runaway loops and premature tool calls are architectural failures the prompt cannot reliably fix. Reach for a schema change or a hard guardrail first when the failure is structural, and reserve prompt restructuring (Chapter 26) for genuine instruction-following degradation under real context growth.

24.5 Designing a stress test

Research topic 25 in this project's own research ledger — "Configuration Combination Testing Functions As Architectural Stress Testing" — makes an argument directly applicable here: "running identical queries through different flag combinations exposed hidden dependencies, undocumented assumptions, and unsupported execution paths that normal testing never exercised." The same discipline applies to context size specifically. `run_combinations.py`, built for flag-combination testing, is the right shape of tool to adapt for a context-size sweep: the same query, run repeatedly with deliberately inflated context (via padding history with additional retrieved chunks or synthetic prior turns) at increasing sizes, watching for the specific point where output quality changes.

A minimal version of this stress test needs only three runs: one comfortably inside the green zone from Section 24.2, one straddling the 16K yellow-zone boundary, and one deliberately pushed toward the red zone. Comparing the three answers side by side — not just for correctness, but for the specific failure signatures from Section 24.4 (fabricated claims, ignored output-format rules, or unbounded tool-call repetition) — locates a pipeline's actual failure cliff far more precisely than trusting a vendor's advertised window size.

24.6 Reading the failure cliff from real logs

The `[CTXSIZE]` line Chapter 23.5 introduced is the exact instrument a stress test like Section 24.5's needs: it reports both the estimated and actual prompt-token count for every call, meaning a stress-test run's own log output already contains the data needed to correlate a specific token count with whatever output-quality change is observed at that point in the run. No separate profiling tooling is required — the same logging built for cost tracking doubles as the instrumentation for a reliability boundary search.

```
[CTXSIZE] iter=N msgs=M chars=C prompt_tokens=P cum_prompt_tokens=T
# read alongside the generated answer for iteration N:
#   T inside Figure 24.1's green zone → treat the answer as trustworthy
#   T inside the yellow zone          → re-check citations against chunks
#   T inside the red zone              → treat as unverified, prefer a
retry
```

Annotating a stress-test transcript this way — pairing each `[CTXSIZE]` line with a manual pass/fail judgment on that iteration's output — is what turns Section 24.5's three-run sweep from an anecdote into a reusable calibration table for a specific deployment's actual model, prompt, and corpus, rather than a one-time observation that goes stale the moment any of those three things changes.

24.7 What upgrading buys you

ADR-005's recommendation of `llama-3.3-70b-versatile` "for production agentic use" is not simply about the larger 32,768-token output ceiling from Chapter 23's table — it is specifically about "much better tool-use and instruction-following," which shifts where the yellow and red zones from Figure 24.1 actually begin. A 70B model does not eliminate the lost-in-the-middle effect, but it pushes the degradation onset further out, buying a pipeline more genuinely reliable context before compression becomes strictly necessary rather than merely prudent.

This is the upgrade path ADR-018's two-LLM design already anticipated: `judge_llm` — the role most sensitive to faithful, careful reasoning over potentially long compressed context — is the first candidate for a 70B swap, while the main agent `llm`, whose calls stay comparatively short and orchestration-focused, can often remain on the faster 8B model without hitting the failure modes this chapter catalogued.

The cost side of that trade is real and worth stating plainly rather than treating the 70B model as a strictly dominant choice. ADR-005 is explicit that the 70B model is "slower and more expensive" — a hybrid architecture is a bet that the reliability gained on the judge and merge roles, where a wrong verdict propagates directly into what the user sees, is worth more than the latency and cost saved by running every role on the faster model. For a role like retrieval-query generation, where a single bad query simply costs one wasted retrieval call rather than a wrong final answer, that trade often runs the other way.

It is also worth being clear about what upgrading does not fix. None of the three failure modes cataloged in Section 24.4 disappear on a bigger or more capable model — they merely relocate. A 70B model with better instruction-following still needs `MAX_ITERATIONS` as a backstop, because the runaway-loop failure mode is a missing guardrail, not an instruction-following gap the model could reason its way out of on its own. Treating a model upgrade as a substitute for the architectural fixes Section 24.4 already identified would leave BUG-F013's failure mode intact, just delayed to a higher iteration count before the rate limit fires.

None of the three failure modes this chapter named are solved by a bigger context window alone. A runaway loop with a 128K-token ceiling instead of a smaller one just takes longer to hit its rate limit. The next two chapters take the opposite approach: rather than asking for a larger, more forgiving window, they ask what a small model specifically needs — in its state tracking and in its prompt structure — to stay reliable inside the window it already has.

That question is worth asking precisely because a hybrid architecture is not always available. A deployment constrained to a single model — by cost, by latency requirements, or simply by what a provider offers — cannot buy back reliability with a bigger model on the roles that need it most. For that deployment, understanding exactly why the 8B model struggles, not just that it struggles, is the only lever left, and that is the question Chapter 25 takes up directly.